REAL-TIME AI-DRIVEN AGILE REQUIREMENT HARMONIZATION

R26-SE-039

Project Proposal Report

A.G Jinad Induvidwa – IT22251114

B.Sc. (Hons) Degree in Information Technology Specialized in Software Engineering

Department of Information Technology

Sri Lanka Institute of Information Technology Sri Lanka

May 2026

DECLARATION

DECLARATION

I declare that this is my own work, and this proposal does not knowingly incorporate any material previously submitted for a degree or diploma at any other university or higher education institution, nor does it contain any material that has been previously published or written by another person apart from where proper acknowledgment is given in the text.

Name	Student ID	Signature
Jinad Induvidwa A.G	IT22251114	

The above candidate is conducting research for their undergraduate Dissertation under my supervision.

Name	Date	Signature
Mrs. Suriya Kumari	14/03/2026	

ABSTRACT

In Agile software development, translating spoken stakeholder requirements into technical documentation often suffers from information decay and late-discovered logical contradictions. Current transcription tools and Agile systems lack the domain-specific intelligence to simultaneously evaluate acoustic confidence, detect semantic conflicts, and provide immediate visual feedback to resolve contradictions while the meeting is still in progress. To address this gap, this research proposes a real-time requirement harmonization system that captures live audio to transcribe and assign stakeholder dialogue using speaker diarization. The system evaluates paralinguistic cues and uses Large Language Model (LLM) sliding window analysis to detect contradictions, triggering a visual “Red Warning” for immediate team consensus.

The proposed solution utilizes WebRTC for audio capture, Faster-Whisper for technical transcription, and Pyannote Audio for diarization. It also employs OpenSMILE for paralinguistic extraction, Llama 3 with ChromaDB for semantic memory, and LangGraph for logic orchestration. Ultimately, the system is expected to automatically generate structured, conflict-free JSON user stories that are pre-evaluated for testability using the INVEST criteria. Technically, it aims to achieve a Real-Time Factor (RTF) of less than 1.0 and a domain-specific Word Error Rate (WER) under 15%.

Keywords: Agile requirements, Speaker diarization, Paralinguistic analysis, Semantic conflict detection, User story generation.

ACKNOWLEDGEMENT

I would like to express my deepest gratitude to my supervisor, Mrs. Suriyaa Kumari , for her invaluable guidance, continuous encouragement, and constructive feedback throughout the development of this research proposal for the project titled " **REAL-TIME AI-DRIVEN AGILE REQUIREMENT HARMONIZATION.**". Her expertise and insights have been instrumental in shaping the direction and methodology of this research. I also extend my sincere thanks to the Sri Lanka Institute of Information Technology for providing the academic foundation, facilities, and resources necessary to pursue my B.Sc. (Hons) Degree in Information Technology Specialization in Software Engineering. Finally, I would like to thank my peers, friends, and family for their unwavering support and motivation during the preparation of this proposal.

TABLE OF CONTENT

DECLARATION 2
ABSTRACT 3
ACKNOWLEDGEMENT 4
TABLE OF CONTENT 5
LIST OF TABLES 6
LIST OF FIGURES 7
LIST OF ABBREVIATIONS 8
LIST OF APPENDICES 9
1. INTRODUCTION 10
1.1 Background and Context 10
1.2 Literature Review 11
1.3 Research Gap 11
1.4 Research Value 12
1.5 Product Value 12
2. OBJECTIVE 14
2.1 Main Objective 14
2.2 Specific Objectives 14
3. METHODOLOGY 15
4. HIGH-LEVEL SYSTEM ARCHITECTURE 16
5. USER REQUIREMENTS 17
6. COMMERCIALIZATION PLAN 19
7. BUDGET AND JUSTIFICATION 21
8. WORK BREAKDOWN STRUCTURE (WBS) 22
9. GANTT CHART 23
10. REFERENCES LIST 24
11. APPENDICES 26

LIST OF TABLES

Table 1: Comparison of existing methods and the proposed system 12
Table 2: Estimated budget table 21

LIST OF FIGURES

Figure 1: Individual Component System Architecture 16
Figure 2: Overall System Architecture 16
Figure 3: Work Breakdown Structure 22
Figure 4: Gantt chart 23
Figure 5: Sequence Diagram 26
Figure 6: UI Mockups 27

LIST OF ABBREVIATIONS

Description	Abbreviations
ASR	Automatic Speech Recognition
INVEST	Independent, Negotiable, Valuable, Estimable, Small, Testable
NER	Named Entity Recognition
NLP	Natural Language Processing
RAG	Retrieval-Augmented Generation
RTF	Real-Time Factor
WER	Word Error Rate

LIST OF APPENDICES

Appendix 1: Turnitin.....28

1. INTRODUCTION

1.1 Background and Context

Agile software development relies heavily on the accurate and rapid translation of stakeholder requirements into actionable technical documentation [1]. In modern remote and hybrid work environments, these requirements are typically discussed in live meeting settings where significant information decay often occurs between the verbal agreement and the written user story [1], [2]. The core problem addressed by this research is the informational decay and logical inconsistency that happens when ephemeral stakeholder speech is manually converted into technical requirements. This late discovery of stakeholder contradictions leads to costly rework and delayed QA cycles, affecting entire Agile teams, including Product Owners, Business Analysts, and QA Engineers [14]. Furthermore, QA engineers who manually design and write test cases from these flawed or ambiguous criteria often miss important edge cases, resulting in lower test coverage and hidden defects in both user interfaces and backend logic. The capture of software requirements is a critical phase where bad requirements can lead to significant rework and project failure, a problem exacerbated by manual bottlenecks that are time-consuming and prone to subjective interpretation [14].

While general-purpose transcription tools and Large Language Models (LLMs) are increasingly used for summarization, they exhibit significant limitations in specialized Agile environments. Standard Automatic Speech Recognition (ASR) systems lack the domain-specific intelligence required to accurately navigate complex software engineering jargon, such as CI/CD or Kubernetes. Moreover, existing systems focus primarily on recording text without evaluating the internal logic or the acoustic confidence of the speakers. They routinely discard implicit paralinguistic cues such as hesitations, long pauses, and jitter which often correlate with speaker uncertainty, leading to overconfident but flawed requirements. Existing methodologies also separate the transcription process from the validation process, requiring manual review against quality frameworks like INVEST and failing to provide an immediate visual feedback loop to resolve stakeholder contradictions while the meeting is still in progress.

To address these gaps, this research proposes a real-time requirement harmonization system that utilizes speaker diarization, paralinguistic analysis, and LLM prompt chaining to detect semantic conflicts and evaluate requirement quality as they are being spoken. The solution integrates a real-time acoustic layer using Faster-Whisper and Pyannote Audio for high-speed transcription and streaming speaker diarization. Simultaneously, a semantic intelligence and conflict detection layer utilizes Llama 3 and a ChromaDB vector store in a sliding window analysis to cross-reference new statements against session history to identify logical contradictions. When a semantic conflict or high acoustic uncertainty is detected, the system triggers a "Visual Warning Protocol" (Red Warning) on a live dashboard, prompting immediate team consensus. Finally, the system employs Retrieval-Augmented Generation (RAG) and prompt chaining to synthesize multi-speaker transcripts into structured JSON requirements , which then serve as the foundation for generating framework-agnostic test scripts.

This project aligns directly with the United Nations Sustainable Development Goals, specifically SDG 9 (Industry, Innovation, and Infrastructure) by fostering innovation in software engineering infrastructure through a novel, AI-driven pipeline. It also supports SDG 8 (Decent Work and Economic Growth) by automating repetitive, manual QA tasks, thereby promoting higher productivity and a more streamlined work environment for Agile

development teams. Academically, the research falls squarely within the Software Engineering research cluster, focusing on Agile methodologies, software quality assurance, and requirements engineering, while holding a strong secondary alignment with Artificial Intelligence by leveraging Natural Language Processing (NLP), LLMs, and paralinguistic speech analysis.

1.2 Literature Review

Recent literature highlights significant advancements in isolated technologies relevant to Agile requirement capture and natural language processing. Foundational research into robust automatic speech recognition (ASR) via large-scale weak supervision and optimized inference engines like Faster-Whisper have enabled high-speed technical transcription [4], [3], [5]. Concurrently, speaker diarization pipelines such as pyannote.audio facilitate the identification of individual stakeholders during live discussions [6]. In the realm of semantic intelligence, the advent of Retrieval-Augmented Generation (RAG) has allowed systems to synthesize complex domain knowledge, while frameworks like LangChain support highly composable LLM applications [16], [17]. Commercially, existing products and services heavily utilize these standard ASR and LLM summarization tools to transcribe general meetings; however, these services predominantly treat meeting transcription and technical requirement validation as distinct, disconnected processes [1].

Despite these technological advancements, existing systems exhibit critical limitations when applied specifically to Agile software engineering workflows. Standard ASR tools and commercial meeting summarizers fail to capture implicit paralinguistic cues such as hesitations, long pauses, or jitter which can be extracted by specialized tools like openSMILE (and supplemented via audio analysis libraries such as librosa) to gauge speaker uncertainty [7], [22]. Furthermore, current market solutions lack integrated, real-time sliding-window cross-referencing capable of continuously comparing new statements against previously established session memory (e.g., vector stores such as ChromaDB) to detect logical contradictions [9], [16]. Consequently, Agile teams are left to manually validate delayed transcripts against quality standards like the INVEST framework [11]. Ultimately, existing approaches lack the capability to provide an immediate visual feedback loop (such as a “Red Warning”) necessary for real-time stakeholder conflict resolution before requirements are finalized [1].

1.3 Research Gap

Existing Agile support tools and Automatic Speech Recognition (ASR) systems are insufficient because they focus primarily on basic transcription without evaluating the internal logic or the acoustic confidence of speakers [1], [4]. Currently, the transcription of stakeholder meetings and the validation of those requirements against quality frameworks like INVEST are treated as isolated, asynchronous processes [11]. This disconnect leads to late conflict discovery, where logical contradictions and implicit ambiguities often signaled by discarded paralinguistic cues like long pauses or hesitations are only identified long after the meeting has concluded [7]. Consequently, Agile teams remain vulnerable to high costs associated with rework, as conflict discovery happens late in the lifecycle [14].

To address this critical gap, this research introduces an integrated pipeline combining role-based diarization, real-time semantic conflict detection, and paralinguistic risk scoring into a unified system [6], [7], [16]. The key creative contribution is the immediate “Visual Warning

Protocol" ("Red Warning" mechanism). By utilizing an LLM and vector memory to cross-reference live statements, the system can instantly detect contradictions and alert the team to resolve the conflict in-session, improving requirements quality before downstream QA automation [9], [10], [16].

Table 1: Comparison of existing methods and the proposed system

Feature	**Standard ASR Tools**	**LLM Summarizers**	**Proposed System**
Real-Time Technical Transcription	Limited	No	Yes (Faster-Whisper)
Role-Based Diarization	Basic	Yes	Yes (BA vs. PO)
Acoustic Risk Analysis (Paralinguistics)	No	No	Yes (OpenSMILE/Librosa)
Real-Time Semantic Conflict Detection	No	Limited	Yes (Llama 3/GPT-4o)
INVEST Quality Scoring	No	No	Yes (Automated JSON)

1.4 Research Value

This research delivers significant academic and practical value by introducing a unified, real-time Agile requirement harmonization pipeline that is not available in current meeting transcription or Agile tooling. Existing systems typically perform transcription and summarization, but do not validate logic, uncertainty, and contradictions while the meeting is in progress. The proposed solution integrates acoustic intelligence, paralinguistic risk scoring, semantic conflict detection, and automated INVEST-aligned JSON user story generation to reduce ambiguity and late rework. It advances research in Requirements Engineering and AI-driven multimodal analysis by enabling real-time conflict detection and immediate stakeholder alignment.

1.5 Product Value

The system provides strong product value by acting as a "virtual scrum master" that captures requirements accurately, detects contradictions instantly, and generates test-ready user stories. It reduces downstream rework, accelerates QA preparation, and improves requirement clarity for Product Owners, Business Analysts, Developers, and QA Engineers.

Similar Available Products

Tool	Feature	Limitations
Notion AI / ClickUp AI	Documentation support	Works after the meeting, no ASR confidence analysis, no live risk detection
Otter.ai	Transcription, meeting notes	No real-time risk detection, limited ASR confidence analysis
Fireflies.ai	Meeting transcription, action items	Works post-meeting, lacks live risk detection
Fathom	Call summaries, note-taking	No live ASR confidence, no proactive risk alerts

Proposed Feature Differences

The proposed system differentiates itself through the following unique capabilities:

- Real-time semantic conflict detection with immediate “Red Warning” visual alerts
- Role-based diarization aligned to Agile roles (PO, BA, QA, Developer) for accurate attribution and accountability
- Paralinguistic risk scoring (hesitation, jitter, long pauses) to flag uncertain or ambiguous requirements
- Technical-jargon-optimized Faster-Whisper ASR with a target domain WER < 15%
- Sliding-window vector memory to cross-reference new statements against meeting history
- Automatic generation of structured JSON user stories using RAG and LLM orchestration
- Automated INVEST scoring and testability rating before saving finalized stories
- Privacy-first design: ephemeral audio processing with no persistent raw-audio storage

2. OBJECTIVE

2.1 Main Objective

The main objective of this research is to design, develop, and evaluate a real-time, AI-driven Agile requirement harmonization pipeline by the project's conclusion in Dec 2026 , which accurately transcribes multi-speaker meetings to instantly detect logical contradictions via a visual "Red Warning" protocol and automatically generates INVEST-validated JSON user stories. This objective will be quantitatively measured and considered successfully achieved when the integrated system demonstrates a transcription Real-Time Factor (RTF) of less than 1.0 , maintains a Word Error Rate (WER) strictly below 15% for technical software engineering jargon , and successfully executes semantic conflict alerts within a 5-second latency window, thereby providing a highly reliable and automated foundation for downstream QA testing workflows.

2.2 Specific Objectives

Acoustic Processing and Diarization: To design and optimize a domain-specific Automatic Speech Recognition (ASR) pipeline using Faster-Whisper and Pyannote Audio that accurately transcribes and diarizes multi-stakeholder agile meetings. This component will be measured by achieving a Word Error Rate (WER) of less than 15% for technical software engineering jargon and maintaining a Real-Time Factor (RTF) below

Paralinguistic Risk Assessment: To evaluate the correlation between paralinguistic voice cues, such as hesitations exceeding 2 seconds or measurable jitter, and requirement ambiguity. This objective is achieved by utilizing these extracted cues to continuously calculate and assign an "Acoustic Risk Score" to specific spoken requirements during the meeting.

Real-Time Semantic Conflict Detection: To implement a semantic intelligence framework utilizing Llama 3 and a ChromaDB vector store to cross-reference live transcripts against established session memory. The success of this objective will be measured by the system's ability to detect logical contradictions and return a "Warning/No Warning" status within a strict 5-second processing window.

Immediate Visual Feedback (Red Warning): To construct a real-time web-based dashboard that integrates a "Visual Warning Protocol". This objective focuses on immediately triggering a red visual alert on the user interface whenever a semantic conflict or platform mismatch is detected, thereby forcing rapid stakeholder consensus.

Automated Generation and Quality Scoring: To develop a Retrieval-Augmented Generation (RAG) and prompt chaining module that synthesizes resolved meeting transcripts into formal, structured JSON User Stories. This objective is fulfilled when the system successfully generates the stories and automatically assigns a measurable Testability Score based on the INVEST criteria prior to database storage.

3. METHODOLOGY

This project adopts a Design Science Research (DSR) approach, as it focuses on the creation, implementation, and evaluation of a novel technological artifact [18].

- **Streaming Diarization and ASR**: Raw audio captured via WebRTC is piped through WebSockets to an acoustic layer that performs real-time transcription and diarization concurrently [2], [19], [4], [6] .
- **Paralinguistic Feature Extraction**: Algorithms extract acoustic markers (e.g., jitter, silence duration) to calculate an "Acoustic Risk Score" [7], [22] .
- **Sliding Window Semantic Analysis:** As transcripts are generated, an LLM cross-references new segments against a vector-based session memory (ChromaDB) to identify logical contradictions in near real-time [8], [9], [16] .
- **RAG and Prompt Chaining:** Upon conflict resolution, the system synthesizes corrected transcript data using RAG to output structured JSON requirements/user stories [16], [15] .
- **FastAPI & React**: FastAPI manages asynchronous WebSocket streams for low-latency transport, while React powers the live dashboard UI [20], [21].

Data Collection and Ethical Considerations

Data is acquired dynamically by capturing live audio streams directly from participants' microphones via a custom WebRTC interface on the web dashboard. Because this involves recording multi-stakeholder business meetings, data privacy is highly prioritized. The system ensures ethical handling by processing audio streams entirely in ephemeral memory. Raw audio is never persistently stored; only the finalized, approved JSON User Stories are saved to the system's database, ensuring confidentiality and compliance with privacy standards.

Validation Strategy and Performance Metrics

The system's performance and accuracy will be validated against strict quantitative metrics:

- **Real-Time Factor (RTF) and Latency:** The system must achieve an RTF of less than 1.0, ensuring the live transcript appears on the dashboard with under 3 seconds of latency [12].
- **Conflict Detection Speed:** The semantic conflict module must process new segments and trigger a warning status within a 5-second window.
- **Word Error Rate (WER):** The ASR pipeline will be benchmarked to maintain a WER of less than 15%, specifically evaluated on domain-specific software engineering jargon [13].

4. HIGH-LEVEL SYSTEM ARCHITECTURE

Figure 1: Individual Component System Architecture

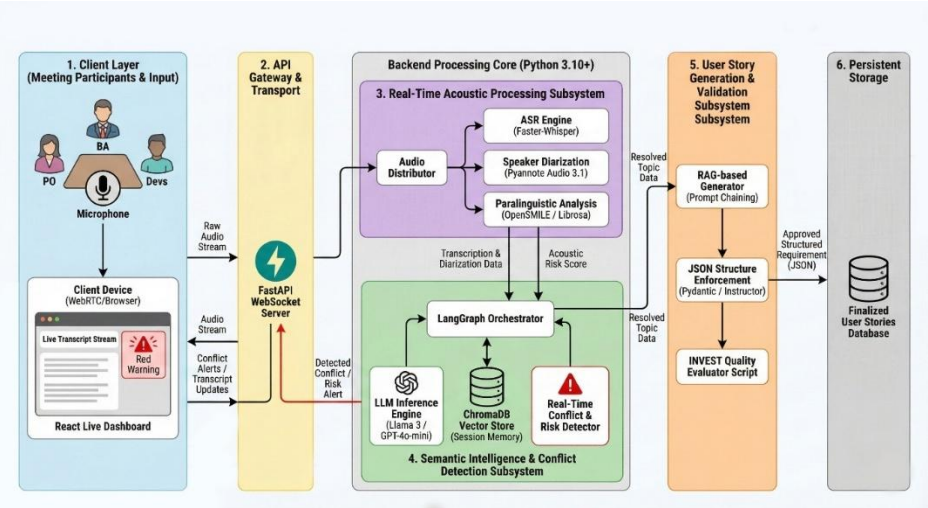
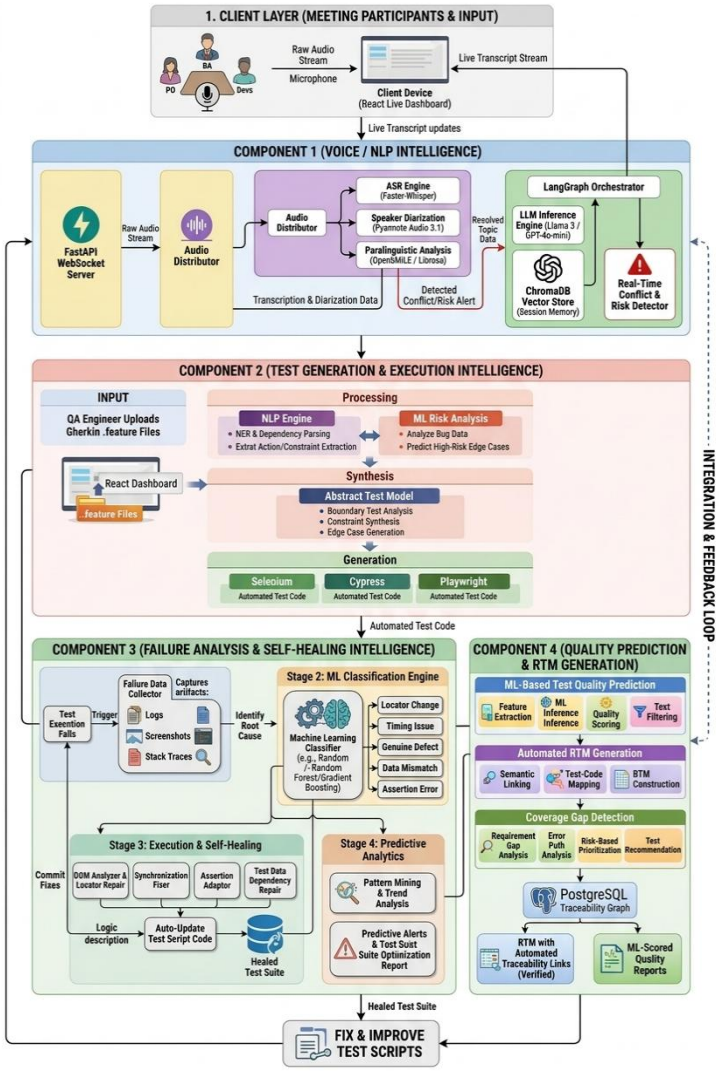


Figure 2: Overall System Architecture



5. USER REQUIREMENTS

Stakeholder Identification The system is designed to serve multiple key stakeholders within Agile software development environments. The primary stakeholders include:

- Product Owners (POs) and Business Analysts (BAs): The primary drivers of the meetings who articulate the features and constraints, and whose speech is monitored for logical consistency.
- QA Engineers: The primary beneficiaries who require structured, unambiguous, and testable user stories to formulate test cases.
- Software Developers: Team members who rely on the harmonized requirements to accurately build the software.
- Scrum Masters / Project Managers: Stakeholders who utilize the platform as a "virtual scrum master" to ensure meetings remain productive and conflicts are resolved on the spot.

Functional Requirements

1. The system must capture real-time audio from multiple speakers via a web-based WebRTC interface and stream it to a backend server.
2. The system must perform real-time speech-to-text transcription utilizing a fine-tuned Faster-Whisper model capable of recognizing software engineering jargon.
3. The system must execute speaker diarization to identify and label distinct roles (e.g., "Product Owner", "Business Analyst") in the live transcript.
4. The system must detect logical contradictions in real-time by cross-referencing new spoken statements against the session's history stored in a ChromaDB vector database.
5. The system must trigger an immediate "Visual Warning Protocol" (Red Warning) on the dashboard whenever a semantic conflict or platform mismatch is detected.
6. The system must analyze paralinguistic voice cues, such as hesitations or jitter, to calculate and assign an "Acoustic Risk Score" to specific requirements.
7. The system must generate structured User Stories in JSON format using Retrieval-Augmented Generation (RAG) once a discussion topic is resolved.
8. The system must automatically evaluate the generated user stories against the INVEST criteria and assign a "Testability Score" prior to saving them to the database.

Non-Functional Requirements

- Performance: The system must achieve a Real-Time Factor (RTF) of less than 1.0, ensuring that live transcripts appear on the user dashboard with a latency of under 3 seconds. Additionally, the conflict detection module must process new transcript

segments and return a warning status within 5 seconds to remain relevant to the ongoing conversation.

- Accuracy: The transcription pipeline must maintain a Word Error Rate (WER) of less than 15% specifically for domain-specific technical terms.
- Reliability: The WebSocket connection facilitating the audio stream must be robust, automatically attempting to reconnect in the event of network instability.
- Usability: The frontend dashboard must feature an intuitive interface that clearly distinguishes between the live transcript stream and the system alerts (Red Warnings) to prevent user confusion during live meetings.
- Security and Privacy: The system must ensure strict data privacy by processing all audio streams in ephemeral memory; only the finalized, mutually approved User Stories may be stored in the persistent database.

6. COMMERCIALIZATION PLAN

Target Market and Customer Persona

- **Target Market:** The primary market consists of distributed software development teams and software agencies operating in remote or hybrid work environments. A secondary expansion market includes enterprise clients needing secure automated documentation, as well as the legal and medical sectors (for depositions and consultations).
- **Customer Persona:** The ideal customer personas include Project Managers, virtual Scrum Masters, Business Analysts, and QA Leads who are responsible for ensuring accurate requirements and managing software quality. Secondary personas include Agile consultancies and organizational psychologists interested in meeting patterns.

Value Proposition

The system acts as a "virtual scrum master" that ensures requirements are captured accurately across different time zones. By detecting logical conflicts and ambiguity *during* the meeting rather than after, the system directly prevents "bad requirements," significantly reducing costly project rework and providing a high return on investment (ROI) for software agencies. Additionally, it fosters inclusivity by supporting neurodiverse team members through the conversion of ephemeral speech into structured, visual text with explicit conflict warnings.

Revenue Model

The project will generate revenue through multiple streams:

- **SaaS Model:** Implementing a "QA-as-a-Service" subscription model for software teams.
- **Data Monetization:** Creating a secondary revenue stream by providing aggregated, anonymized data insights on Agile meeting patterns (e.g., "teams that pause more produce better code") to Agile consultancies.

Pricing Strategy

The platform will utilize a tiered subscription pricing strategy.

- **Freemium Tier:** Basic transcription and diarization features will be offered for free to attract users.
- **Premium Tier:** Customers will pay a premium subscription fee to access the platform's "Predictive Intelligence," which includes advanced features like Red Warnings, automated INVEST scoring, and RAG-based User Story generation.

Competitive Advantage

Unlike standard ASR tools or LLM summarizers, this system's primary competitive advantage is its ability to perform Real-Time Conflict Detection *during* the meeting. It is unique in simultaneously combining role-based diarization, paralinguistic risk scoring (detecting

hesitations and jitter), and real-time semantic conflict detection into a single pipeline. Furthermore, it provides an immediate visual feedback loop (Red Warning) to resolve contradictions instantly, a feature lacking in current market solutions.

Intellectual Property Considerations

The core intellectual property of this project resides in its proprietary "Acoustic + Semantic Intelligence" engine. This engine, which combines multi-speaker diarization with real-time conflict detection, holds significant value and will be protected and packaged for exclusive B2B licensing opportunities with major enterprise communication platforms

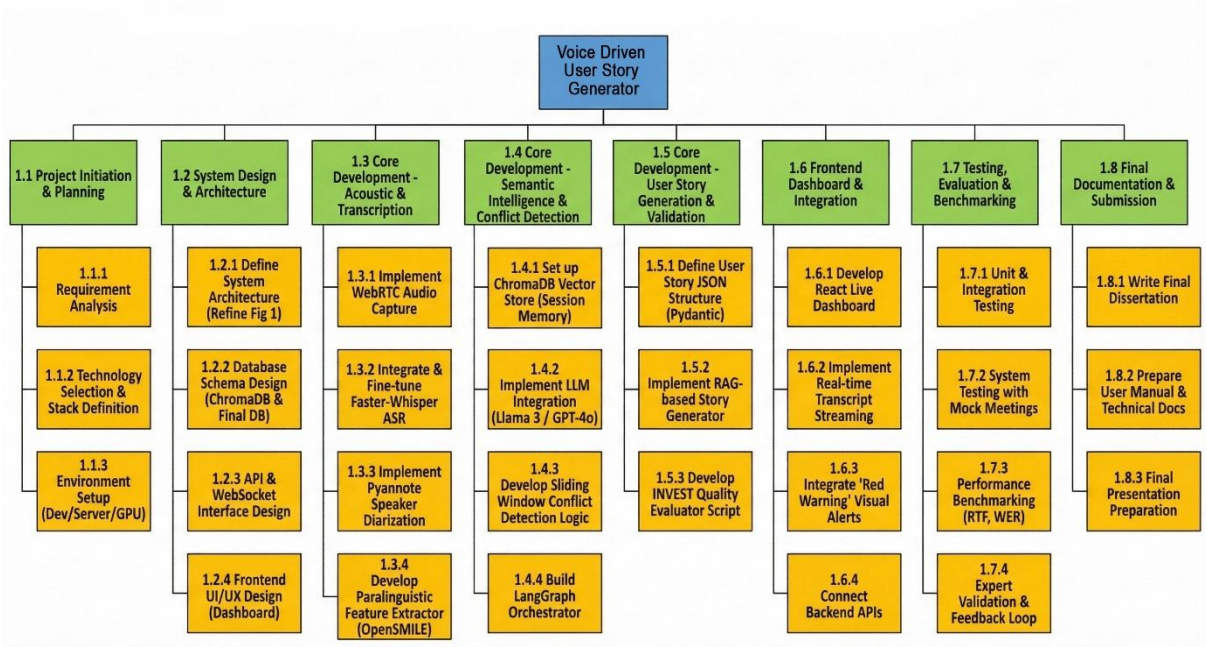
7. BUDGET AND JUSTIFICATION

Table 2: Estimated budget table

Category	Description of Expense	Estimated Cost (Rs.)	Justification
Hardware costs	High-Fidelity Microphone	10,000	A high-quality microphone is required for accurate WebRTC audio testing.
Software licenses	Development IDEs & Open-Source Tools	0	Utilizing free student licenses for development environments (e.g., JetBrains) and free open-source models (Faster-Whisper, Pyannote, local Llama 3, ChromaDB).
Cloud services	Cloud GPU Subscriptions (e.g., Colab Pro)	15,000	Covers approximately 4–5 months of Google Colab Pro (~$10/month) needed to fine-tune the Whisper model for domain-specific Agile software engineering jargon.
Cloud services	Cloud Inference APIs (OpenAI/Cloud LLMs)	25,000	Estimated API usage for GPT-4o-mini or similar models during high-logic conflict validation, semantic processing, and rigorous fallback testing over a 6-month period.
Cloud services	Web Hosting & Deployment (AWS/GCP)	12,000	Hosting the FastAPI backend, WebSocket server, and React frontend on a cloud instance (e.g., DigitalOcean Droplet or AWS EC2) for live testing and external expert validation.
Human effort cost	Expert Validation Honorarium	10,000	Compensation for two industry professionals (Senior BAs/QA Leads) at Rs. 5000 each to thoroughly review generated User Stories and validate the INVEST testability scoring.
Total Budget		72,000	

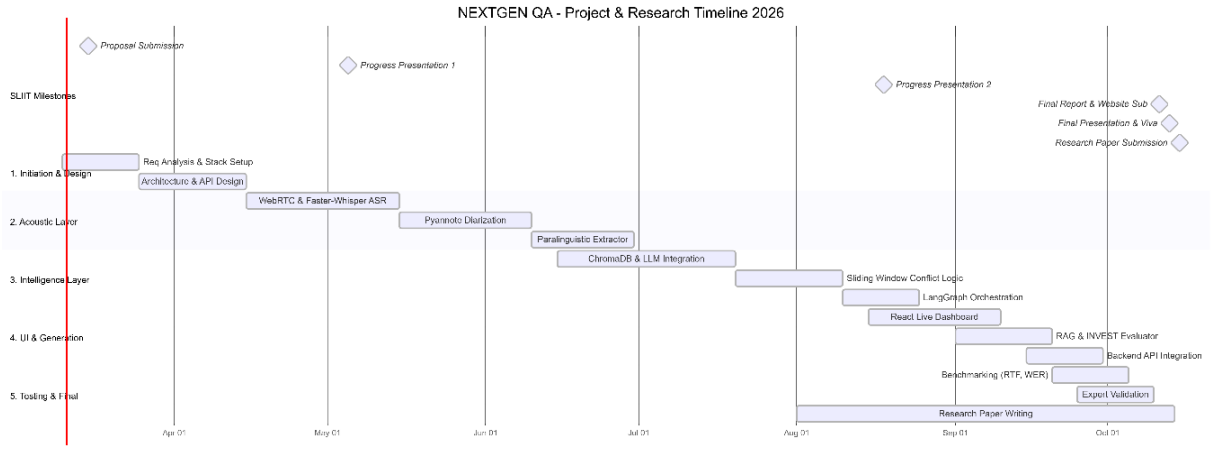
8. WORK BREAKDOWN STRUCTURE (WBS)

Figure 3: Work Breakdown Structure



9. GANTT CHART

Figure 4: Gantt chart



10. REFERENCES LIST

[1] Z. Hoy and M. Xu, “Agile Software Requirements Engineering Challenges-Solutions—A Conceptual Framework from Systematic Literature Review,” Information, vol. 14, no. 6, art. 322, Jun. 2023, doi: 10.3390/info14060322.

[2] W3C, “WebRTC: Real-Time Communication in Browsers,” W3C Recommendation, Mar. 13, 2025. [Online]. Available: https://www.w3.org/TR/webrtc/

[3] SYSTRAN, “faster-whisper: Faster Whisper transcription with CTranslate2,” GitHub repository. [Online]. Available: https://github.com/SYSTRAN/faster-whisper

[4] A. Radford et al., “Robust Speech Recognition via Large-Scale Weak Supervision,” in Proc. ICML, PMLR, vol. 202, pp. 28492–28518, 2023. [Online]. Available: https://proceedings.mlr.press/v202/radford23a.html

[5] OpenNMT, “CTranslate2: Fast inference engine for Transformer models,” GitHub repository. [Online]. Available: https://github.com/OpenNMT/CTranslate2

[6] H. Bredin, “pyannote.audio 2.1 speaker diarization pipeline: principle, benchmark, and recipe,” in Proc. Interspeech, 2023. [Online]. Available: https://www.isca-archive.org/interspeech_2023/bredin23_interspeech.html

[7] F. Eyben, M. Wöllmer, and B. Schuller, “openSMILE – The Munich Versatile and Fast Open-Source Audio Feature Extractor,” in Proc. ACM MM, 2010, pp. 1459–1462, doi: 10.1145/1873951.1874246.

[8] A. Grattafiori et al., “The Llama 3 Herd of Models,” arXiv:2407.21783, Nov. 2024. [Online]. Available: https://arxiv.org/abs/2407.21783

[9] Chroma, “Chroma: Open-source search and retrieval database for AI applications,” GitHub repository. [Online]. Available: https://github.com/chroma-core/chroma

[10] langchain-ai, “LangGraph: Build resilient language agents as graphs,” GitHub repository. [Online]. Available: https://github.com/langchain-ai/langgraph

[11] B. Wake, “INVEST in Good Stories, and SMART Tasks,” XP123, Aug. 17, 2003. [Online]. Available: https://xp123.com/invest-in-good-stories-and-smart-tasks/

[12] Microsoft, “Embedded speech performance evaluations: Real-time factor (RTF),” Microsoft Learn, Feb. 25, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/ai-services/speech-service/embedded-speech-performance-evaluations

[13] NIST, “ASR Metrics: Word Error Rate (WER) definition,” TREC SDR Track slides, 2000. [Online]. Available: https://trec.nist.gov/pubs/trec9/sdrt9_slides/tsld017.htm

[14] B. Boehm and V. R. Basili, “Software Defect Reduction Top 10 List,” Computer, vol. 34, no. 1, pp. 135–137, Jan. 2001, doi: 10.1109/2.962984.

[15] M. Cohn, User Stories Applied: For Agile Software Development. Boston, MA, USA: Addison-Wesley, 2004.

[16] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in Advances in Neural Information Processing Systems, vol. 33, pp. 9459–9474, 2020. [Online]. Available: https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html

[17] LangChain, “LangChain overview (Python),” Documentation. [Online]. Available: https://docs.langchain.com/oss/python/langchain/overview

[18] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design Science in Information Systems Research,” MIS Quarterly, vol. 28, no. 1, pp. 75–105, Mar. 2004, doi: 10.2307/25148625.

[19] I. Fette and A. Melnikov, “The WebSocket Protocol,” RFC 6455, Dec. 2011, doi: 10.17487/RFC6455.

[20] S. Ramírez, “WebSockets - FastAPI Advanced User Guide,” FastAPI Documentation. [Online]. Available: https://fastapi.tiangolo.com/advanced/websockets/

[21] React Team, “React Documentation,” [Online]. Available: https://react.dev/

[22] B. McFee et al., “librosa: Audio and music signal analysis in python,” in Proc. SciPy, 2015. [Online]. Available: https://proceedings.scipy.org/articles/Majora-7b98e3ed-003.pdf

11. APPENDICES

Appendix A: Survey Instruments

This survey is designed for the Senior Agile Team Including Business Analysts and QA Leads participating in the expert validation phase of the generated User Stories.

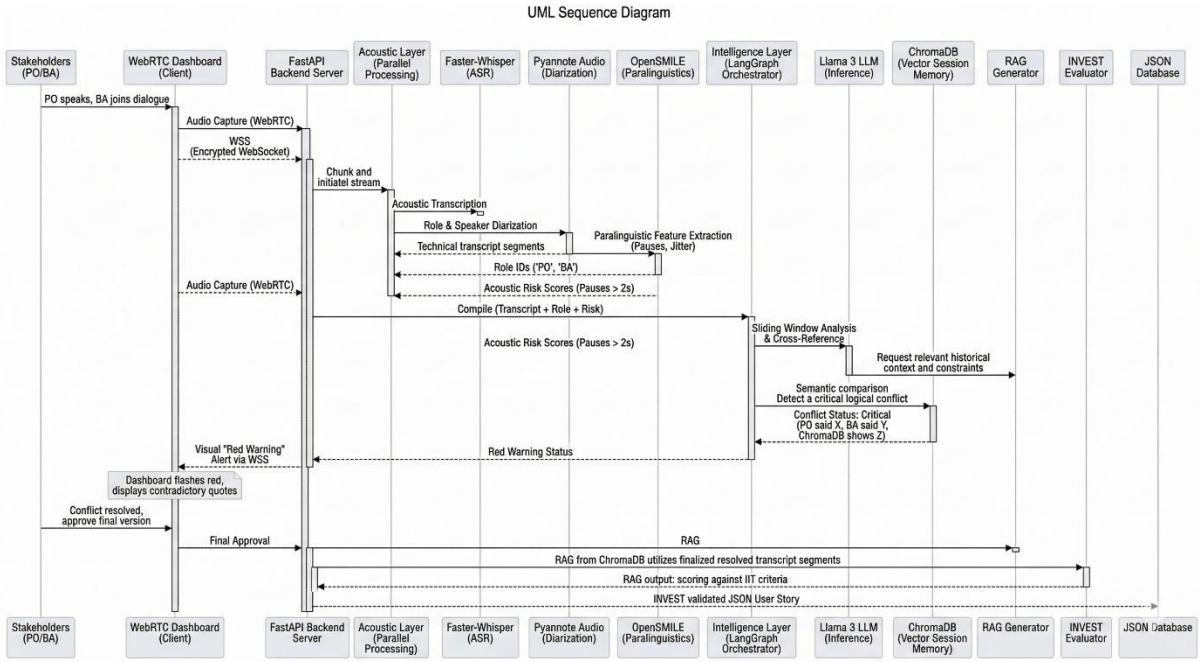
System Evaluation Questionnaire

(Likert Scale 1-5: 1 = Strongly Disagree, 5 = Strongly Agree)

1. The transcription accurately captured domain-specific software engineering jargon (e.g., "CI/CD", "Kubernetes").
2. The speaker diarization correctly attributed dialogue to the appropriate roles (BA, PO, Developer).
3. The "Red Warning" visual alert correctly identified logical contradictions during the live meeting.
4. The generated JSON User Story accurately reflects the final consensus reached by the team.
5. The automated INVEST testability score aligns with my professional assessment of the User Story.

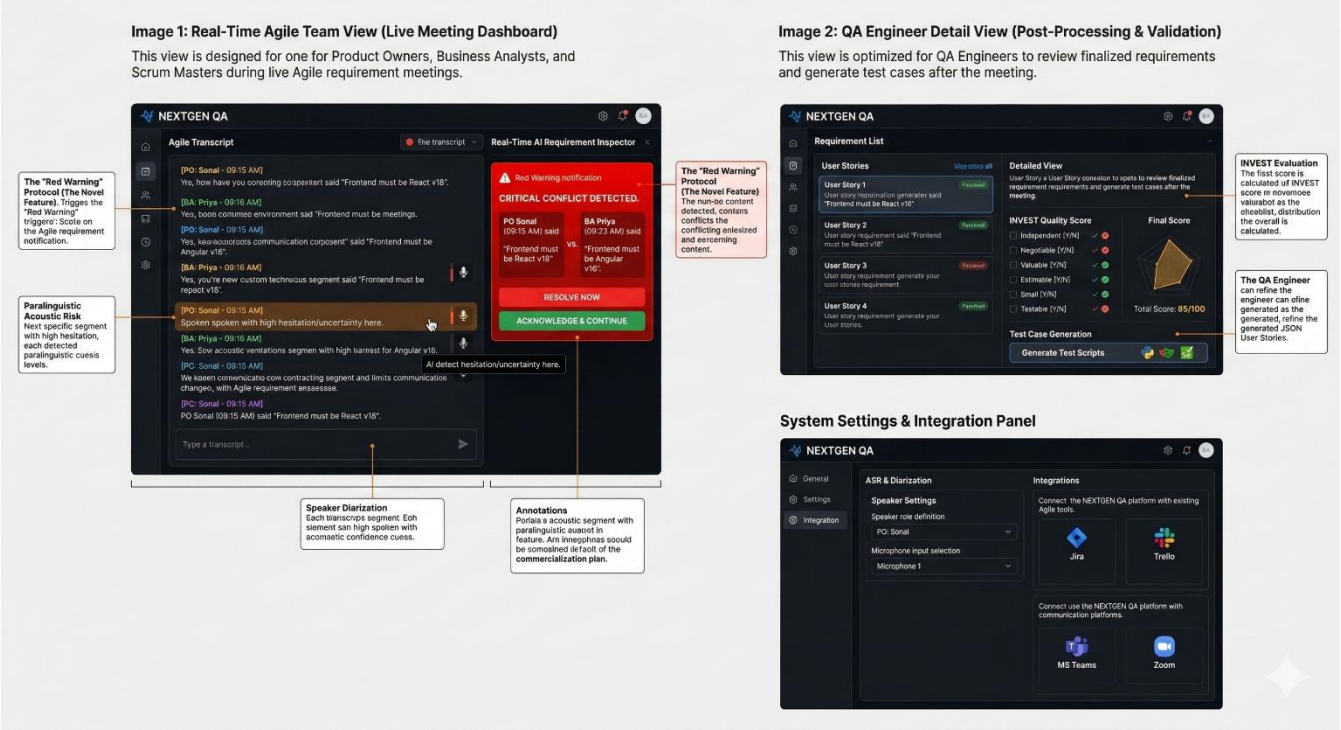
Appendix B: Detailed Diagrams

Figure 5: Sequence Diagram



Appendix C: Preliminary UI Mockups

Figure 6: UI Mockups



- **Dashboard Layout:** A web interface built in React.
- **Left Panel (Live Stream):** Displays the continuous, scrolling transcript with speaker tags (e.g., **[Product Owner - 10:14 AM]**).
- **Right Panel (System Alerts):** Houses the "Visual Warning Protocol". When a conflict occurs, a bright red alert box appears here explicitly quoting the contradictory statements.
- **Bottom Panel (Output):** A structured data view displaying the finalized JSON User Story and its corresponding INVEST testability score radar chart.

Appendix D: Risk Analysis

Risk Description	Impact	Probability	Mitigation Strategy
High Latency (RTF > 1.0): Real-time transcription lags significantly behind live speech.	High	Medium	Utilize CTranslate2 quantization for the Faster-Whisper model and ensure dedicated GPU resources (minimum 6GB VRAM) for local inference.
LLM Hallucinations: The RAG generator invents constraints not discussed in the meeting.	High	Medium	Utilize CTranslate2 quantization for the Faster-Whisper model and ensure dedicated GPU resources (minimum 6GB VRAM) for local inference.
Noisy Audio Input: Background noise disrupts	Medium	High	Implement WebRTC-level noise suppression and echo cancellation on

the Pyannote diarization pipeline.			the client side before WebSocket transmission.

Appendix E: Ethical Documents

Participant Consent Form (Draft)

Project: NEXTGEN QA – VOICE DRIVEN TEST CASE GENERATION AND SELF-HEALING
By participating in this Agile requirements meeting, you consent to having your voice recorded and processed by an automated AI transcription and semantic analysis system.

- **Data Privacy Guarantee:** All audio streams are processed in ephemeral memory. Raw audio recordings and paralinguistic voice data are **deleted immediately** post-processing and are never persistently stored on any server.
- **Stored Data:** Only the finalized, mutually agreed-upon text (JSON User Stories) will be saved to the project database for downstream QA automation.
- **Right to Withdraw:** You may request the system be muted or stopped at any point during the meeting.

Appendix F: Originality Report

Appendix 1: Turnitin

The following report summarizes the similarity index for the submitted proposal "Voice Driven Test Case Generation and Self-Healing for Agile QA Automation"